

PROJECT 31A — APPLICATION SECURITY BIBLE (DOCUMENT 8)

Version 1.0.0

**Classification: CONFIDENTIAL — FOR INTERNAL
DEVELOPMENT AND DUE DILIGENCE ONLY**

1. SECURITY ARCHITECTURE OVERVIEW

Project 31A implements a defense-in-depth security model to ensure multi-tenant data isolation, secure credential management, fail-safe API routing, and complete operational audit trails. The platform's design is governed by the core principle: **"AI proposes. Deterministic systems execute. Security controls restrict."**

The platform implements security barriers at multiple levels:

1. **Network Layer:** Restricted ports inside container networks; external traffic must pass through SSL-terminated API Gateways.
 2. **FastAPI Route Level:** Custom middleware stack verifying JWT tokens, checking tenant scopes, checking rate limits, and recording audit logs.
 3. **Services & Activities Level:** Mandatory kill switch validations and credentials vault access checks.
 4. **Database Level:** PostgreSQL Row-Level Security (RLS) policies and append-only database logs.
-

2. PRODUCTION SAFETY GUARDS

To prevent misconfigurations in production environments, `backend/src/seo_platform/main.py:L72-L95` executes four hard validation checks during server boot. If any check fails in an environment where `APP_ENV=production` is set, the application terminates immediately.

2.1 Use of Mock Providers Guard

- **Check:** `settings.use_mock_providers` must be `False`.
- **Reason:** Prevents workers from simulating API calls (e.g. Hunter, Ahrefs, SendGrid) inside live campaigns, which would cause fake prospect data to persist.

2.2 Dev Authentication Bypass Guard

- **Check:** `settings.dev_auth_bypass` must be `False`.
- **Reason:** Dev mode allows all requests without valid JWT authorization by supplying a headers fallback. Enforcing this check prevents unauthenticated root access to API scopes.

2.3 App Debug Mode Guard

- **Check:** `settings.app_debug` must be `False`.
- **Reason:** Debug mode prints detailed tracebacks and exception variables on HTTP error responses, which could leak database schemas or credentials.

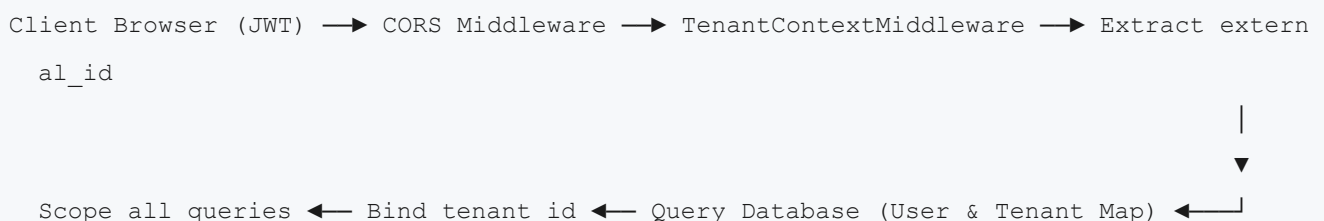
2.4 Encryption Key Entropy Guard

- **Check:** Runs `validate_encryption_key_entropy(settings.encryption_master_key)`.
- **Reason:** Validates that the master key used to encrypt the credential vault contains sufficient entropy (minimum 32-byte key size, high randomness). If the key is a default string or too short, boot halts to prevent insecure encryption configurations.

3. AUTHENTICATION & ACCESS CONTROL

Authentication is managed via external identity providers (**Clerk** or **Auth0**) and integrated into the FastAPI routing layer.

3.1 Authentication Pipeline Flow



1. The client browser obtains a JWT bearer token upon successful authentication.
 2. The request hits the API where the token is parsed by `TenantContextMiddleware`.
 3. The middleware verifies the JWT signature against the provider's JSON Web Key Sets (JWKS).
 4. The user's `external_id` (Clerk ID) is extracted from the JWT claims.
 5. The application queries the local `users` table to map the `external_id` to the internal `user_id` and the user's active `tenant_id`.
 6. The `tenant_id` is bound to the database session context.
-

4. AUTHORIZATION & RBAC POLICY MATRIX

Access rights are governed by a strict Role-Based Access Control (RBAC) policy. The system defines roles and permissions in `backend/src/seo_platform/models/tenant.py` under the `ROLE_PERMISSIONS` dictionary.

Permission / Action	Super Admin	Tenant Admin	Manager	SEO Analyst	Outreach Specialist	Report Analyst	Client
<code>manage_tenant_s</code>	Yes	No	No	No	No	No	No
<code>view_audit_log</code>	Yes	No	No	No	No	No	No
<code>manage_kill_switches</code>	Yes	No	No	No	No	No	No
<code>manage_users</code>	Yes	Yes	No	No	No	No	No
<code>manage_billing</code>	Yes	Yes	No	No	No	No	No
<code>launch_campaign</code>	Yes	Yes	Yes	No	No	No	No
<code>approve_outreach</code>	Yes	Yes	Yes	No	No	No	No
<code>create_keyword_cluster</code>	Yes	Yes	Yes	Yes	Yes	No	No
<code>manage_outreach_threads</code>	Yes	Yes	Yes	No	Yes	No	No
<code>generate_reports</code>	Yes	Yes	Yes	Yes	Yes	Yes	No
<code>view_own_reports</code>	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Enforcement Check: Custom API endpoints utilize the `@require_permission(permission_name)` decorator which evaluates the active user's permissions array before executing route handlers.

5. CREDENTIAL VAULT & ENCRYPTION INFRASTRUCTURE

Sensitive credentials, including Ahrefs API keys, Hunter keys, and citation directory login details, are stored encrypted in the database.

- **Models:** `ProviderKey` and `DirectoryCredential`.
- **Cipher:** AES-256-GCM (Galois/Counter Mode) via the Python cryptography library.

- **Key Derivation:** The key is derived from the `ENCRYPTION_MASTER_KEY` environment variable.
- **Initialization Vector (IV):** A unique, random 12-byte IV is generated for every write operation, ensuring identical passwords generate different ciphertext values.
- **Integrity Tag:** AES-GCM appends a 16-byte authentication tag to the ciphertext, validating that the credentials have not been tampered with or modified.

6. SRE OPERATIONS CONTROLS: KILL SWITCHES

To handle incidents (e.g. LLM hallucinations, email provider suspensions, or scraping detection blocks), SREs can trigger operational kill switches via `/api/v1/kill-switches/`.

- **Switches:** `email_sending` and `llm_inference`.
- **Implementation:** Stored as Redis string keys with tenant context scoping: `killswitch:{switch_name}:{tenant_id}`.
- **Check pattern:**

```
if await kill_switch_service.is_blocked("email_sending", tenant_id):
    logger.warn("kill_switch_active_halting_email_dispatch", tenant_id=tenant_id)
    raise KillSwitchActiveException("Email dispatch has been administratively paused.")
```

- **Guarantees:** Temporal activities execute this check prior to executing side-effects.

7. API SURFACE PROTECTION (SEC-FIX-003)

In Phase P1, the audit identified a critical security risk: when running in development environments (which was the default setup), all 684 API routes were exposed via `/openapi.json` and `/docs` to unauthenticated callers. This provided attackers with a complete map of the API surface.

- **Fix (SEC-FIX-003):** FastAPI documentation (`/docs`, `/redoc`, and `/openapi.json`) is disabled globally in production. In non-production environments, it remains disabled by default. It can only be enabled by setting the environment variable `ENABLE_OPENAPI_DOCS=true` in non-production environments.

8. HTTP SECURITY HEADERS

The application mounts `SecurityHeadersMiddleware` on FastAPI to inject security headers into every response:

- `X-Content-Type-Options: nosniff` : Prevents browsers from MIME-sniffing responses away from declared content-types.
 - `X-Frame-Options: DENY` : Prevents clickjacking attacks by blocking the dashboard from being rendered inside frames or iframes.
 - `Content-Security-Policy (CSP)` : Restricts script executions, styling, and frame sources to trusted origins (Clerk, domain API endpoint).
 - `Strict-Transport-Security (HSTS)` : Enforces SSL communication for browsers.
-

9. AUDIT TRAILS & COMPLIANCE

The `audit_log` table contains detailed records of all security-relevant operations:

- **Immutable Log:** Custom database triggers block UPDATE or DELETE operations on the `audit_log` table, rendering it functionally append-only.
- **State Changes Logging:** Includes `before_state` and `after_state` JSON snapshots of modified database rows to enable review of user actions.
- **Partitioning:** The table is partitioned by month using PostgreSQL `PARTITION BY RANGE (created_at)` to support high-throughput operations.